Employee Attrition Analysis and Predictive Modelling for HR Decision Support

Employee attrition increases recruitment cost, reduces productivity, and disrupts team stability. This projects analyzes HR data to identify key drivers of attrition and builds predicitve models to support proactive retention actions. The focus is on actionable insights HR teams can operationalize

Section 1 - Imports + Load data

In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df=pd.read_excel ("IBM Dataset for project.xlsx")
print ("Shape of dataset:", df.shape)
df.head()
```

```
Shape of dataset: (1470, 35)
```

Out[6]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Educat
0	41	Yes	Travel_Rarely	1102	Sales	1	
1	49	No	Travel_Frequently	279	Research & Development	8	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	
4	27	No	Travel_Rarely	591	Research & Development	2	

5 rows × 35 columns

Section 2 - Quick Audit

In [8]:
```
df.head(3)
```

Out[8]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Educat
0	41	Yes	Travel_Rarely	1102	Sales	1	
1	49	No	Travel_Frequently	279	Research & Development	8	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	

3 rows × 35 columns

Section 3 - Data Types + Missing Values

In [10]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1470 non-null   int64
 1   Attrition                 1470 non-null   object
 2   BusinessTravel            1470 non-null   object
 3   DailyRate                 1470 non-null   int64
 4   Department                1470 non-null   object
 5   DistanceFromHome          1470 non-null   int64
 6   Education                 1470 non-null   int64
 7   EducationField            1470 non-null   object
 8   EmployeeCount             1470 non-null   int64
 9   EmployeeNumber            1470 non-null   int64
 10  EnvironmentSatisfaction   1470 non-null   int64
 11  Gender                    1470 non-null   object
 12  HourlyRate                1470 non-null   int64
 13  JobInvolvement            1470 non-null   int64
 14  JobLevel                  1470 non-null   int64
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   int64
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   int64
 19  MonthlyRate               1470 non-null   int64
 20  NumCompaniesWorked        1470 non-null   int64
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   int64
 24  PerformanceRating         1470 non-null   int64
 25  RelationshipSatisfaction  1470 non-null   int64
 26  StandardHours             1470 non-null   int64
 27  StockOptionLevel          1470 non-null   int64
 28  TotalWorkingYears         1470 non-null   int64
 29  TrainingTimesLastYear     1470 non-null   int64
 30  WorkLifeBalance           1470 non-null   int64
 31  YearsAtCompany            1470 non-null   int64
 32  YearsInCurrentRole        1470 non-null   int64
 33  YearsSinceLastPromotion   1470 non-null   int64
 34  YearsWithCurrManager      1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

In [11]:
```
df.isnull().sum()
```

Out[11]:
```
Age                         0
Attrition                   0
BusinessTravel              0
DailyRate                   0
Department                  0
DistanceFromHome            0
Education                   0
EducationField              0
EmployeeCount               0
EmployeeNumber              0
EnvironmentSatisfaction     0
Gender                      0
HourlyRate                  0
JobInvolvement              0
JobLevel                    0
JobRole                     0
JobSatisfaction             0
MaritalStatus               0
MonthlyIncome               0
MonthlyRate                 0
NumCompaniesWorked          0
Over18                      0
OverTime                    0
PercentSalaryHike           0
PerformanceRating           0
RelationshipSatisfaction    0
StandardHours               0
StockOptionLevel            0
TotalWorkingYears           0
TrainingTimesLastYear       0
WorkLifeBalance             0
YearsAtCompany              0
YearsInCurrentRole          0
YearsSinceLastPromotion     0
YearsWithCurrManager        0
dtype: int64
```

Data quality shows whether missing values exist and confirms variable types for preprocessing decisions

Section 4 - Target Variable (Attrition) Overview

In [14]:
```
df['Attrition'].value_counts()
```

Out[14]:
```
Attrition
No     1233
Yes     237
Name: count, dtype: int64
```

In [15]:
```
attrition_rate = df['Attrition'].value_counts(normalize=True) * 100
attrition_rate.round(2)
```

Out[15]:
```
Attrition
No     83.88
Yes    16.12
Name: proportion, dtype: float64
```

Overall attrition rate is around 16%, indicating moderate class imbalance. Because of this, evaluation will prioritize recall and ROC-AUC rather than accuracy alone.

Section 5 - Drop Non-Informative Columns

In [18]:
```
drop_cols=['EmployeeCount', 'Over18', 'StandardHours', 'EmployeeNumber']
df=df.drop(columns=[c for c in drop_cols if c in df.columns])
df.shape
```

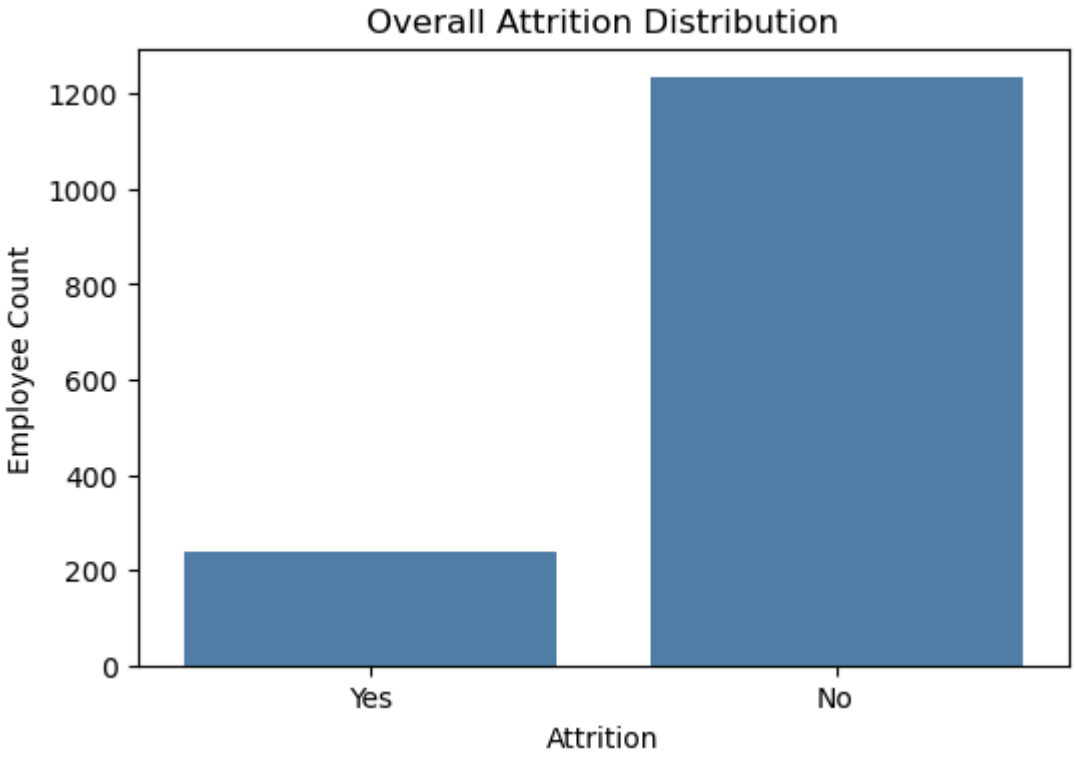
Out[18]:
```
(1470, 31)
```

Constant columns and identifier fields were removed because they do not add explanatory or predictive value and can introduce noise.

Section 6 - EDA starts

6.1 Attrition distribution

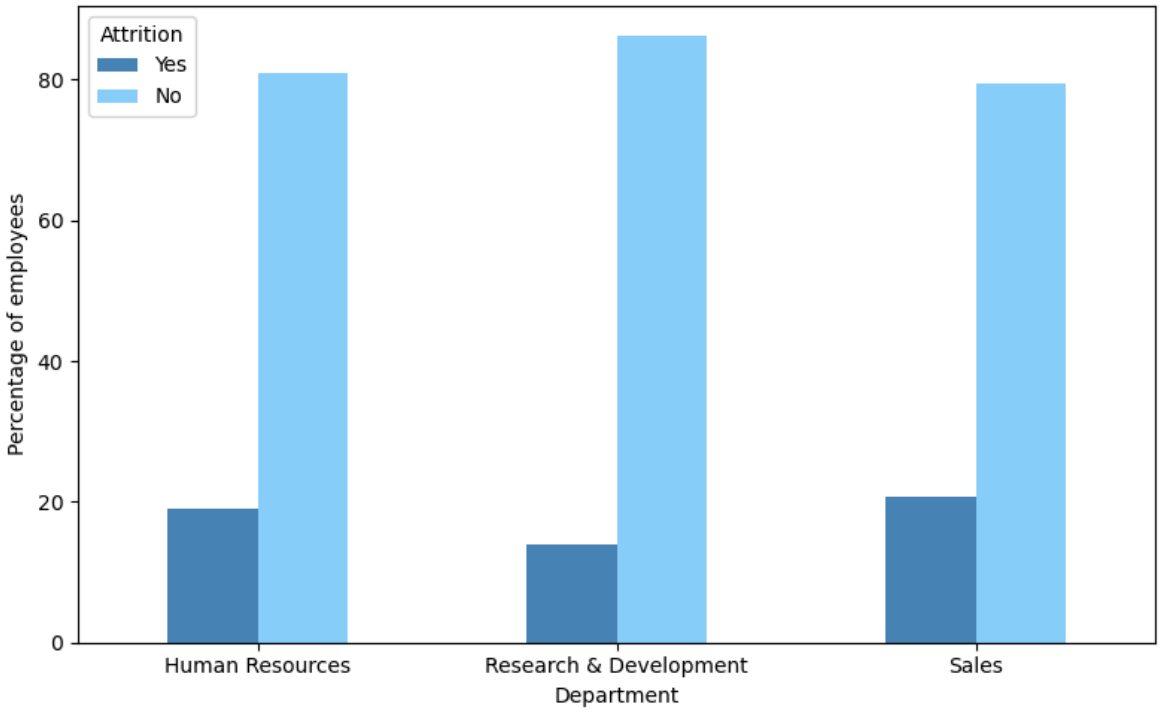
In [22]:
```
plt.figure(figsize=(6,4))
sns.countplot(data=df, x='Attrition', color='steelblue')
plt.title("Overall Attrition Distribution")
plt.xlabel("Attrition")
plt.ylabel("Employee Count")
plt.show()
```



Observation- *Attrition cases are fewer than non-attrition cases.*
HR meaning- *Models must be evaluated beyond accuracy to avoid missing high-risk leavers.*

6.2 Attrition rate by Department (percentage)

In [25]:
```
dept_attr = pd.crosstab(df['Department'], df['Attrition'], normalize='index') *
ax = dept_attr[['Yes','No']].plot(kind='bar', figsize=(8,5), color=['steelblue',
plt.ylabel("Percentage of employees")
plt.legend(title="Attrition")
plt.xticks(rotation=0, ha='center')
plt.tight_layout()
plt.show()
dept_attr[['Yes']].sort_values('Yes', ascending=False).round(2)
```



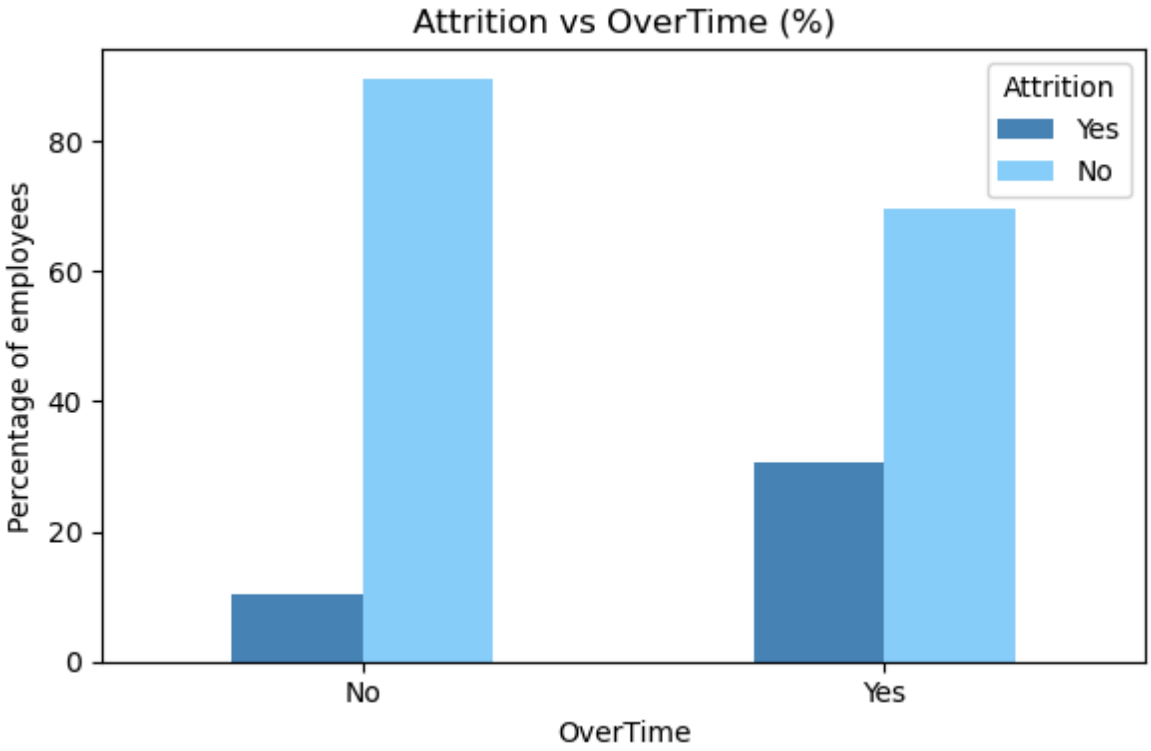
Out[25]:

Attrition	Yes
Department	
Sales	20.63
Human Resources	19.05
Research & Development	13.84

Observation- *Some departments show higher attrition percentages than others.*
HR meaning- *Retention actions should be targeted by department, not applied uniformly.*

6.3 Attrition vs OverTime

In [28]:
```
ot_attr = pd.crosstab(df['OverTime'], df['Attrition'], normalize='index') * 100
ax = ot_attr[['Yes','No']].plot(kind='bar', figsize=(6,4), color=['steelblue', '
plt.title("Attrition vs OverTime (%)")
plt.ylabel("Percentage of employees")
plt.xticks(rotation=0)
plt.legend(title="Attrition")
plt.tight_layout()
plt.show()
ot_attr[['Yes']].round(2)
```



Out[28]:

Attrition	Yes
OverTime	
No	10.44
Yes	30.53

Observation- *Overtime groups often show much higher attrition.*
HR meaning- *Workload intensity and overtime policies are potential retention leavers.*

Section 7 - Tenure Band

7.1 Create TenureBand

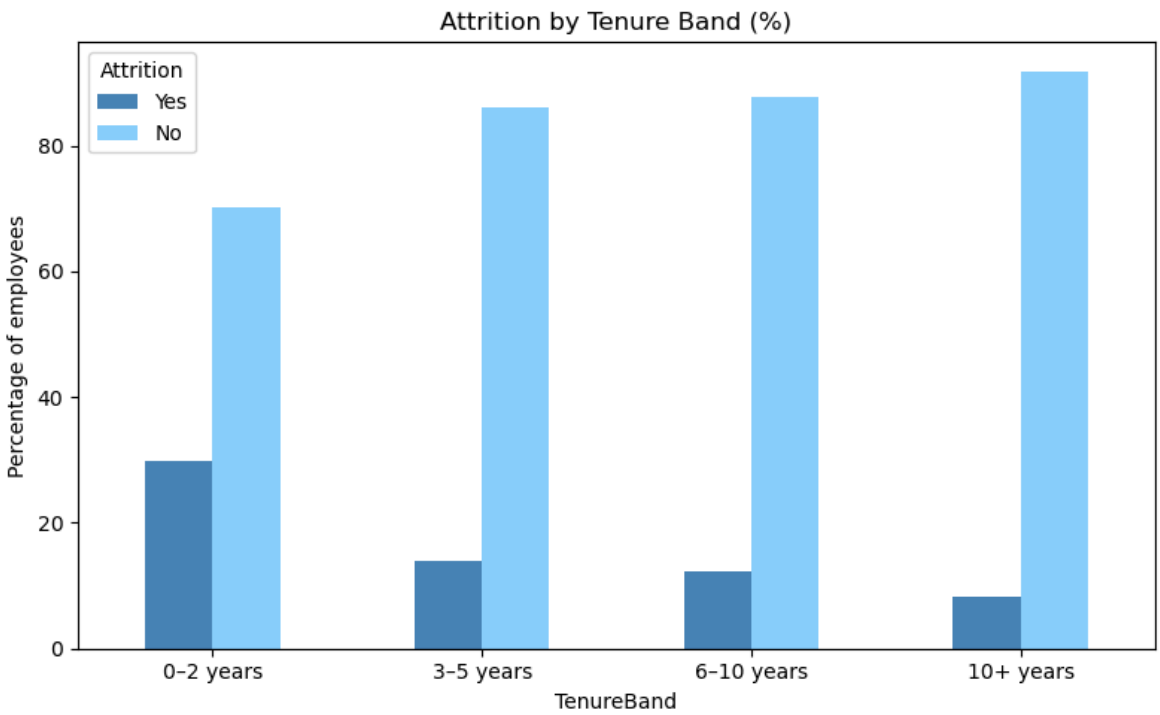
In [32]:
```
bins = [0, 2, 5, 10, df['YearsAtCompany'].max()]
labels = ['0-2 years', '3-5 years', '6-10 years', '10+ years']
df['TenureBand'] = pd.cut(df['YearsAtCompany'], bins=bins, labels=labels, includ
df['TenureBand'].value_counts(dropna=False)
```

Out[32]:
```
TenureBand
6-10 years    448
3-5 years     434
0-2 years     342
10+ years     246
Name: count, dtype: int64
```

7.2 Attrition by TenureBand

In [34]:
```
ten_attr = pd.crosstab(df['TenureBand'], df['Attrition'], normalize='index') * 1
ax = ten_attr[['Yes','No']].plot(kind='bar', figsize=(8,5), color=['steelblue',
plt.title("Attrition by Tenure Band (%)")
plt.ylabel("Percentage of employees")
```

```
plt.xticks(rotation=0)
plt.legend(title="Attrition")
plt.tight_layout()
plt.show()
ten_attr[['Yes']].round(2)
```



Out[34]:

Attrition	Yes
TenureBand	
0–2 years	29.82
3–5 years	13.82
6–10 years	12.28
10+ years	8.13

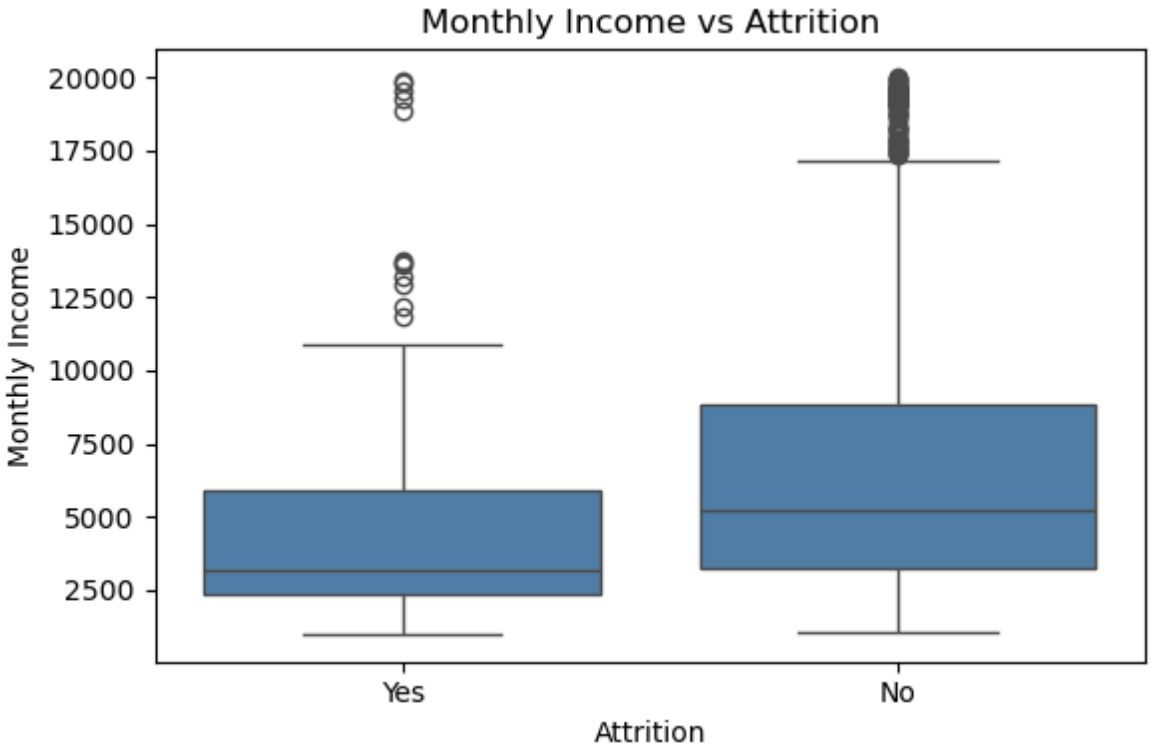
Observation- *Early-tenure employees often have higher attrition.*
HR meaning- *Strengthen onboarding, early career development, and 30/60/90-day check-ins*

Section 8 - Compensation and Satisfaction

8.1 MonthlyIncome vs Attrition

In [38]:

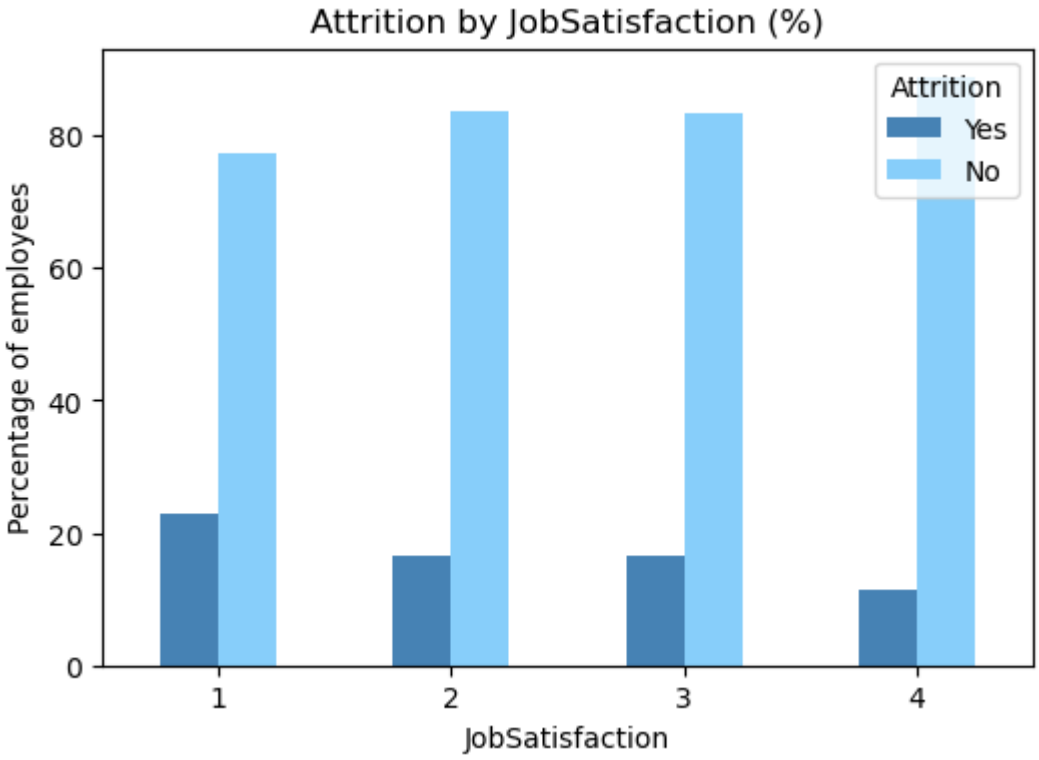
```
plt.figure(figsize=(6,4))
sns.boxplot(data=df, x='Attrition', y='MonthlyIncome', color='steelblue')
plt.title("Monthly Income vs Attrition")
plt.xlabel("Attrition")
plt.ylabel("Monthly Income")
plt.tight_layout()
plt.show()
```

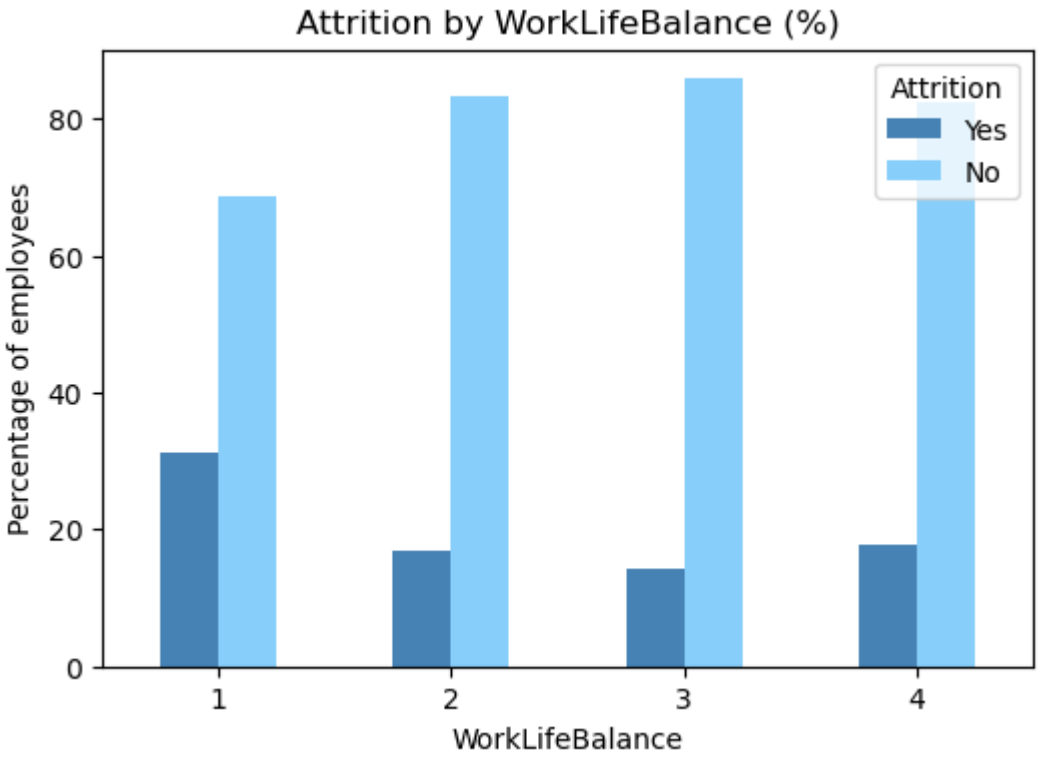


Observation- *Leavers often show lower income distribution.*
HR meaning- *Compensation benchmarking may reduce avoidable exits in valnerable roles.*

8.2 JobSatisfaction and WorkLifeBalance

In [41]:
```
for col in ['JobSatisfaction', 'WorkLifeBalance']:
    temp = pd.crosstab(df[col], df['Attrition'], normalize='index') * 100
    ax = temp[['Yes','No']].plot(kind='bar', figsize=(6,4), color=['steelblue',
    plt.title(f"Attrition by {col} (%)")
    plt.ylabel("Percentage of employees")
    plt.xticks(rotation=0)
    plt.legend(title="Attrition")
    plt.show()
```





Observation- *Lower satisfaction / weaker work-life balance groups show higher attrition.*
HR meaning- *Engagement and wellbeing initiatives should be targeted to groups showing evaluated risk.*

Section 9 - Statistical Testing

1. Chi-square test -> Overtime vs Attrition

2. Chi-square test -> Department vs Attrition

3. t-test -> Monthly Income difference (Yes vs No)

9.1 Chi-Square- Overtime vs Attrition

Why?

Both are categorical variables.

We test whether they are independent.

In [46]:
```
from scipy.stats import chi2_contingency
cont_table_ot = pd.crosstab(df['OverTime'], df['Attrition'])
chi2, p, dof, expected = chi2_contingency(cont_table_ot)
print("Chi-square statistic:", round(chi2,2))
print("p-value:", round(p,4))
```

```
Chi-square statistic: 87.56
p-value: 0.0
```

The chi-square test indicates a statistically association between overtime and attrition (p < 0.05), supporting workload as a key retention factor.

9.2 Chi-Square- Department vs Attrition

In [49]:
```
cont_table_dept = pd.crosstab(df['Department'], df['Attrition'])
chi2_dept, p_dept, dof_dept, expected_dept = chi2_contingency(cont_table_dept)
print("Chi-square statistic:", round(chi2_dept,2))
print("p-value:", round(p_dept,4))
```

```
Chi-square statistic: 10.8
p-value: 0.0045
```

Departmental differences in attrition are statistically significant, indicating that retention strategies should be tailored at department level.

9.3 t-Test- Monthly Income Difference

Why?

MonthlyIncome is numerical.

Attrition is binary.

We test whether income differs between leavers and stayers.

In [52]:
```
from scipy.stats import ttest_ind
leavers = df[df['Attrition'] == 'Yes']['MonthlyIncome']
stayers = df[df['Attrition'] == 'No']['MonthlyIncome']
t_stat, p_val = ttest_ind(leavers, stayers, equal_var=False)
print("t-statistic:", round(t_stat,2))
print("p-value:", round(p_val,4))
```

```
t-statistic: -7.48
p-value: 0.0
```

There is a statistically significant difference in monthly income between employees who left and those who stayed, suggesting compensation may influence attrition.

Section 10 - Predictive Modelling

10.1 Creating a Binary Outcome

In [60]:
```
df = pd.read_excel("IBM Dataset for project.xlsx")
drop_cols = ['EmployeeCount', 'Over18', 'StandardHours', 'EmployeeNumber']
df = df.drop(columns=[c for c in drop_cols if c in df.columns])
df['Attrition'] = df['Attrition'].astype(str).str.strip().str.title()
df['Attrition'] = df['Attrition'].map({'Yes': 1, 'No': 0})
df['Attrition'].isnull().sum(), df['Attrition'].value_counts()
```

Out[60]:
```
(0,
 Attrition
 0    1233
 1     237
 Name: count, dtype: int64)
```

The attrition variable was encoded as a binary outcome (1=Yes, 0=No) to enable classification modelling. Logistic Regression requires numerical targets, and binary encoding allows the model to estimate the probability of employee exit.

In [59]:
```
X = df.drop('Attrition', axis=1)
y = df['Attrition']
X = pd.get_dummies(X, drop_first=True)
print(X.select_dtypes(include=['object']).columns)
```

```
Index([], dtype='object')
```

10.2 Creating Dummy Variables

In [56]:
```
df['Attrition'] = df['Attrition'].map({'Yes': 1, 'No': 0})
X = df.drop('Attrition', axis=1)
y = df['Attrition']
X = pd.get_dummies(X, drop_first=True)
X.shape
```

Out[56]:
```
(1470, 47)
```

Categorical variables were one-hot encoded to enable compatibility with machine learning algorithms. The target variable was converted into binary form for classification modelling

10.3 Train-Test Split

In [63]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

```
Training size: (1176, 47)
Testing size: (294, 47)
```

The dataset was split into training (80%) and Testing (20%) sets using stratified sampling to preserve the class distribution. Stratification ensures that the attrition rate remains consistent across both datasets, enabling reliable model evaluation.

10.4 Scale Numeric Features

In [115…

```
from sklearn.preprocessing import StandardScaler
num_cols= X_train.select_dtypes (include=['int64', 'float64']).columns
scaler= StandardScaler()
X_train[num_cols]=scaler.fit_transform(X_train[num_cols])
X_test[num_cols]=scaler.transform(X_test[num_cols])
```

10.5 Logistic Regression

In [91]:

```
X= df.drop('Attrition', axis=1)
y=df['Attrition']
X= pd.get_dummies(X, drop_first=True)
print("Shape after encoding:", X.shape)
```

```
Shape after encoding: (1470, 44)
```

In [118…

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y)  #keep the y variable same.
print("Train attrition rate:", round(y_train.mean(), 3))
print("Test attrition rate:", round(y_test.mean(), 3))
```

```
Train attrition rate: 0.162
Test attrition rate: 0.16
```

Numeric features were standardised using StandardScaler to ensure stable optimization and prevent dominance of large-scale variables. The scaler was fitted only on training data to prevent data leakage and the same transformation was applied to the test set.

10.6 Model Evaluation

In [148…

```
from sklearn.linear_model import LogisticRegression

log_model = LogisticRegression(
    max_iter=5000,
    class_weight='balanced' ,
    solver='liblinear'
)

log_model.fit(X_train, y_train)
```

Out[148…

```
LogisticRegression
LogisticRegression(class_weight='balanced', max_iter=5000, solver='libl
inear')
```

In [155…
```
y_pred=log_model.predict(X_test)
y_prob=log_model.predict_proba(X_test)[:,1]
```

In [161…
```
from sklearn.metrics import confusion_matrix, classification_report
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, digits=3))
from sklearn.metrics import roc_auc_score
auc=roc_auc_score(y_test, y_prob)
print("ROC-AUC:", round(auc, 3))
```

```
[[192  55]
 [ 17  30]]
              precision    recall  f1-score   support

           0      0.919     0.777     0.842       247
           1      0.353     0.638     0.455        47

    accuracy                          0.755       294
   macro avg      0.636     0.708     0.648       294
weighted avg      0.828     0.755     0.780       294

ROC-AUC: 0.805
```

The Logistic Regression model achieved a ROC-AUC of 0.805, indicating strong discriminatory ability between employees who left and those who stayed. This suggests the model effectively ranks employees by attrition risk and provides meaningful separation between high-risk and low-risk individuals.

10.7 Interpret Coefficients

In [185…
```
import numpy as np
import pandas as pd
coeff_df = pd.DataFrame({
    'Feature': X_train.columns,
    'Coefficient': log_model.coef_[0]})
coeff_df['Odds_Ratio'] = np.exp(coeff_df['Coefficient'])
coeff_df.sort_values(by='Odds_Ratio', ascending=False).head(10)
```

Out[185…

	Feature	Coefficient	Odds_Ratio
23	BusinessTravel_Travel_Frequently	1.653845	5.227037
34	JobRole_Laboratory Technician	1.651108	5.212751
43	OverTime_Yes	1.630711	5.107502
40	JobRole_Sales Representative	1.327519	3.771673
33	JobRole_Human Resources	1.034408	2.813441
24	BusinessTravel_Travel_Rarely	0.969993	2.637925
42	MaritalStatus_Single	0.919396	2.507776
26	Department_Sales	0.695010	2.003729
13	PerformanceRating	0.659265	1.933371
7	JobLevel	0.574121	1.775570

To understand the key drivers of attrition, the model coefficients were converted into odds ratios. Logistic regression estimates the effect of each feature on the log-odds of attrition. By exponentiating the coefficients, odds ratios were obtained to provide interpretable measures of impact. An odds ratio greater than 1 indicates increased attrition risk, while an odds ratio below 1 indicates reduced risk, holding other variables constant.The results show that employees who travel frequently and those working overtime exhibit significantly higher attrition odds compared to their reference groups. Certain job roles also demonstrate elevated risk levels. These findings suggest that workload intensity and role-specific factors may play a critical role in employee turnover.

Section 11 - Risk Segmentation

In [199…

```
df_model = X_test.copy()
df_model['Actual'] = y_test.values
df_model['Predicted_Probability'] = y_prob
df_model['Risk_Category'] = pd.cut(
    df_model['Predicted_Probability'],
    bins=[0, 0.3, 0.6, 1],
    labels=['Low', 'Medium', 'High'])
df_model[['Predicted_Probability','Risk_Category']].head()
```

Out[199…

	Predicted_Probability	Risk_Category
1061	0.307509	Medium
891	0.023980	Low
456	0.480813	Medium
922	0.037298	Low
69	0.650598	High

HR can prioritize high-risk employees for intervention.

Section 12 - Strategic Recommendations

If Overtime is a major driver

1. Review overtime allocation policies

2. Introduce workload monitoring dashboards

3. Add managerial approval for excess overtime

If Early Tenure is high risk

1. 30-60-90 day check-ins

2. Structured mentorship programs

3. Early engagement surveys

If Satisfaction is Key

1. Manager capability training

2. Internal mobility pathways
3. Feedback loop improvements

Section 13 - Conclusion

This analysis identified overtime, tenure, and satisfaction indicators as significant drivers of employee attrition. Logistic Regression demonstrated strong discriminatory performance, and predicted risk probabilities were used to construct a practical risk segmentation framework to support proactive HR interventions.

Exporting a Copy of the Altered Data

In [207…

```
final_df = X_test.copy()
final_df['Actual_Attrition'] = y_test.values
final_df['Predicted_Probability'] = y_prob

final_df['Risk_Category'] = pd.cut(
    final_df['Predicted_Probability'],
    bins=[0, 0.3, 0.6, 1],
    labels=['Low', 'Medium', 'High'])
final_df.to_csv("attrition_risk_output.csv", index=False)
```